1.2. JSP: The first pages

Draft version.
Author: Alexandru Dorobanțiu

We are going to use Jakarta Pages (Jakarta Server Pages, or JSP) for the front-end part of the application.
Disclaimer: At this point I assume you know basic HTML.

Basic info about JSP

JSP (Jakarta server pages) is very similar to PHP (which you may already know), and you may notice that it allows plenty of flexibility.
There are many ways to create pages and templates. What is presented here is one way to work which I find simple to understand.

To start JSP scripting in a page (called JSP scriptlets), we use the syntax **<%** and close with **%>**
In comparison, in PHP we would have used ***<?php*** and ***?>***

Besides Java code, one can write directives, like ***<%@include file="response.jsp" %>*** (notice the @ after tag open). The scriptlets from the pages are compiled.

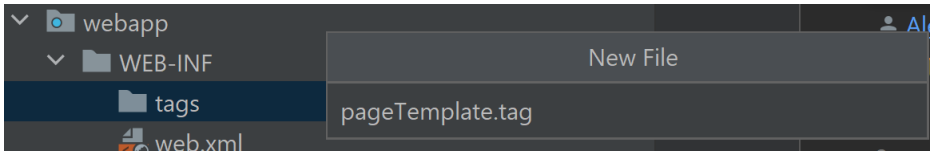
JSP provides XML syntax tags, which are accessible via the namespace ***<jsp:***
Example:

```
<jsp:include page='newspaperColumn.jsp'>
  <jsp:param name="articleId" value=""/>
</jsp:include>
```

Optional Note:
Details about JSP: https://jakarta.ee/specifications/pages/
See more about the JSP syntax and the naming here: https://www.tutorialspoint.com/jsp/jsp_syntax.htm
See the following links to go into more details about templating:
http://wiki4.caucho.com/Java_EE_Servlet_tutorial_:_Using_JSPs_to_create_header,_footer_area,_formatting,_and_basic_CSS_for_bookstore#.2FWEB-INF.2Fpages.2Ftemplate.jsp
https://stackoverflow.com/questions/10529963/what-is-the-best-way-to-create-jsp-layout-template
https://stackoverflow.com/questions/1296235/jsp-tricks-to-make-templating-easier

1. Create a template and a page

Create a new directory under *ParkingLot/src/main/webapp/WEB-INF* and call it **tags**
Create a new file in that folder and call it **pageTemplate.tag**



a. Copy the whole contents of *index.jsp* into the *pageTemplate.tag* file.
b. Replace page with tag

Replace the line

```
<%@page contentType="text/html;charset=UTF-8" pageEncoding="UTF-8"%>
```

with

```
<%@tag description="base page template" pageEncoding="UTF-8"%>
```

This marks that we have a tag instead of a page.

c. Add an attribute

Add a new line below the *tag* definition

```
<%@attribute name="pageTitle"%>
```

Replace the content of the <title> tag with **${pageTitle}**

d. Update the body

Replace the whole content of the *<body>* tag with **<jsp:doBody/>**
The result should look like this:

pageTemplate.tag

```
<%@tag description="base page template" pageEncoding="UTF-8" %>
<%@attribute name="pageTitle" %>

<!DOCTYPE html>
<html>
<head>
    <title>${pageTitle}</title>
</head>
<body>
<jsp:doBody/>
</body>
</html>
```

Next, we want to update the *index.jsp* file to use the template we just created.

a. Include the *tags* folder with a prefix

Add the following line below the *page* definition

```
<%@taglib prefix="t" tagdir="/WEB-INF/tags" %>
```

We can now:

- use the template we created before by opening a tag and using the namespace "t"
- use the *pageTitle* attribute to our *pageTemplate* with the value **Parking Lot**
- add some content in the tag

b. replace the rest of the index.jsp file with a <t:pageTemplate

index.jsp

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@taglib prefix="t" tagdir="/WEB-INF/tags" %>

<t:pageTemplate pageTitle="Parking Lot">
    <h1>Welcome to the Parking Lot</h1>
</t:pageTemplate>
```

Save and Run the project. You should now see the "Welcome to the Parking Lot" message in the browser.

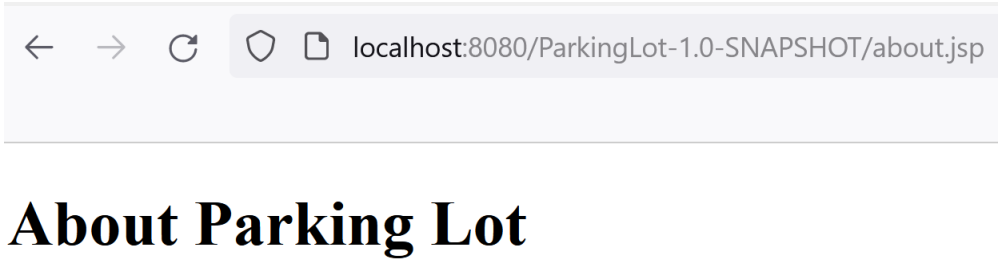
Simple explanation on what we implemented so far

- the content inside the tag is filled in place of the *<jsp:doBody />*
- the attribute *pageTitle* is passed in the attribute variable declared in the template and is accessible within the template via the Jakarta Expression Language (EL) syntax ${....}

> *Optional note:*
> See more about EL here: https://jakarta.ee/specifications/expression-language/5.0/
> Examples of EL: https://jakarta.ee/learn/docs/jakartaee-tutorial/current/web/faces-el/faces-el.html

2. Task - Create an about page

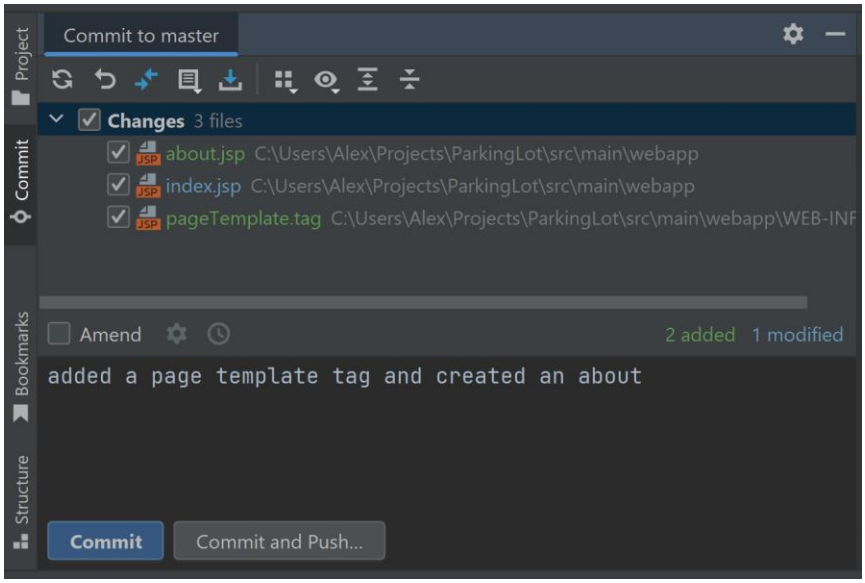
Create another page, like *index.jsp* (and in the same path) and call it **about.jsp**
Change the *pageTitle* attribute to **About Parking Lot** and the content of the *<t:pageTemplate>* tag.
Save the file and access the link in the browser: http://localhost:8080/ParkingLot-1.0-SNAPSHOT/about.jsp



3. Commit the changes

What we achieved so far with the *pageTemplate.tag* is that we now have a single place to modify the basic structure of a page, and leaving the pages to only describe their content.
The templating we used is a form of the decorator pattern (we decorate the content with the template)

Commit changes using the *Commit* panel, make sure all the changes are checked, and add the message "**added a page template tag and created an about**".



Disclaimer: this tutorial does not have the intention to teach you how to use git, nor does it provide the idea of a branching strategy. It is important, though, to learn to commit the changes which are related to one another.